Project Report

Assignment 4 – Computer Networks

Part A – Ping

We performed this assignment using Oracle VirtualBox with Ubuntu 22.04, as required. By default, the VM uses a NAT network adapter, but this interferes with the assignment, especially when testing TTL. Therefore, we use a wired connection and configure the VM to use a Bridged Adapter.

A Makefile is included for convenient compilation of all parts of the assignment. The Makefile allows each program (ping, traceroute, port_scanning) to be compiled separately, or all of them together using the make command, without dependencies between the files:

- make – compiles all programs (ping, traceroute, port_scanning).
- make ping – compiles only the ping program from ping.c.
- make traceroute – compiles only the traceroute program from traceroute.c.
- make port_scanning – compiles only the port_scanning program from port_scanning.c.
- make clean – removes the generated executable files and cleans the directory.

We used various course materials, mainly the code covered in tutorials 5–8.

We begin by writing a simple C program that receives the parameters required by the assignment, validates them, and displays an appropriate error message if they are invalid.

```c
int main(int argc, char **argv) {
    // Step 1: Read command line args
    char *address = NULL; // Where to ping
    long count = -1; // How many pings (-1 = infinite)
    int flood = 0; // Flood mode on/off

    int opt;
    while ((opt = getopt(argc, argv, "a:c:f")) != -1) {
        switch (opt) {

            case 'a':
                address = optarg; // Save destination address
                break;

            case 'c': {
                // Convert count from string to number
                char *end = NULL;
                errno = 0;
                long value = strtol(optarg, &end, 10);

                // Make sure the number is valid and positive
                if (errno != 0 || end == optarg || *end != '\0' || value <= 0) {
                    fprintf(stderr, "Invalid value for -c: %s\n", optarg);
                    return 1;
                }

                count = value;
                break;
            }

            case 'f':
                flood = 1; // No delay between pings
                break;
        }
    }

    // The -a flag is needed
    if (address == NULL) {
        fprintf(stderr, "Error: -a <address> is required\n");
        return 1;
    }

    // Prepare address and socket:
    struct sockaddr_in dest;
    // Clean the structure
    memset(&dest, 0, sizeof(dest));
    dest.sin_family = AF_INET;
    // Convert IPv4 string ("8.8.8.8") into binary form
    if (inet_pton(AF_INET, address, &dest.sin_addr) != 1) {
        fprintf(stderr, "Invalid IPv4 address: %s\n", address);
        return 1;
    }
```

We organized all relevant addresses in the appropriate struct data structures.

Next, we use the Raw Socket code demonstrated in the tutorial and create a raw socket of our own. We open a RAW SOCKET and verify that it was created successfully (administrator privileges are required at runtime, i.e., SUDO):

```c
// Step 2: Create a raw socket for ICMP
int sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
if (sock < 0) {
    perror("socket");
    fprintf(stderr, "Socket didnt run! make sure you use sudo!!!\n");
    return 1;
}

// Print log:
printf("Pinging %s with 64 bytes of data:\n", address);

close(sock);
return 0;
```

Now we need to construct an ICMP message according to what we learned in the course; we refer to the appendix in the assignment:

ICMP Echo Request Header																															
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Type (8 for IPv4)								Code (Always 0)								Checksum (2 Bytes)															
Identifier (2 Bytes)																Sequence Number (2 Bytes)															
Payload																															

To calculate the checksum, we use the function provided in the tutorial code and construct the message using struct icmphdr:

```c
// Step 3: build and send the ICMP msg:

// Allocate buffer for the ICMP packet (header + payload)
unsigned char packet[ICMP_PACKET_SIZE];
// Clear the memory
memset(packet, 0, sizeof(packet));

// Treat the beginning of the buffer as an ICMP header
struct icmphdr *icmp = (struct icmphdr *)packet;

// Set ICMP type to Echo Request (ping)
icmp->type = ICMP_ECHO;
icmp->code = 0;
icmp->un.echo.id = htons((unsigned short)(getpid() & 0xFFFF)); // identifier
icmp->un.echo.sequence = htons(1); // first sequence number

// Fill ICMP payload (56 bytes) with simple data
// This makes the total packet size 64 bytes
for (int i = 0; i < ICMP_PAYLOAD_SIZE; i++) {
    packet[sizeof(struct icmphdr) + i] = (unsigned char)('A' + (i % 26));
}

// Compute checksum over the entire ICMP packet (header + payload)
icmp->checksum = 0;
icmp->checksum = checksum(packet, sizeof(packet));

// Send the packet to the destination (no ports in ICMP, so sockaddr_in is enough)
ssize_t sent = sendto(sock, packet, sizeof(packet), 0,
                      (struct sockaddr *)&dest, sizeof(dest));
if (sent < 0) {
    // Check if sending failed
    perror("sendto");
    close(sock);
    return 1;
}

// Debug message to confirm packet was sent
printf("Sent %zd bytes of ICMP Echo Request\n", sent);

close(sock);
return 0;
```

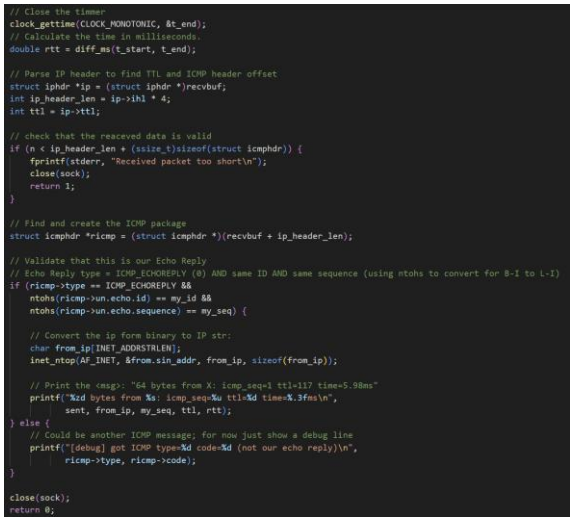
We then build a complete ICMP Echo Request packet in memory (header + payload), calculate its checksum, and send it to the destination through the raw socket using sendto.

At this stage, the start time is measured using CLOCK_MONOTONIC (from time.h). The ICMP Echo Request is sent, and we wait up to 10 milliseconds for a reply using poll. In the event of a timeout, the program terminates and closes according to the assignment requirements.

If a reply is received, we obtain an IP packet (which of course arrives as raw bytes), extract the IP header length and the ICMP header, and verify that it is an Echo Reply matching the request we sent (id and sequence). If the packet does not match our request, we ignore it.

We measure the RTT in milliseconds, as is customary, and display the RTT, the icmp_seq number, and the TTL value to the user.

```c
// Step 3: send and get respond (aka: echo)
// Start counting time for RTT
struct timespec t_start, t_end;
clock_gettime(CLOCK_MONOTONIC, &t_start);

// Send the packet to the destination (no ports in ICMP, so sockaddr_in is enough)
ssize_t sent = sendto(sock, packet, sizeof(packet), 0,
                      (struct sockaddr *)&dest, sizeof(dest));
if (sent < 0) {
    // Check if sending failed
    perror("sendto");
    close(sock);
    return 1;
}

// Wait up to 10 seconds for a reply using poll()
// Use poll for calling us when the socket is readable
struct pollfd pfd;
pfd.fd = sock;
pfd.events = POLLIN;

int pr = poll(&pfd, 1, TIMEOUT_MS);
if (pr == 0) {
    // timeout
    printf("Request timeout (no reply within 10 seconds)\n");
    close(sock);
    return 0; // program ends here in this step
}
if (pr < 0) {
    // inernal error
    perror("poll");
    close(sock);
    return 1;
}

// Receive the raw IP packet (it contains IP header + ICMP message)
unsigned char recvbuf[RECV_BUF_SIZE];
struct sockaddr_in from;
socklen_t fromlen = sizeof(from);
ssize_t n = recvfrom(sock, recvbuf, sizeof(recvbuf), 0,
                     (struct sockaddr *)&from, &fromlen);
if (n < 0) {
    perror("recvfrom");
    close(sock);
    return 1;
}
```



```
// Close the timmer
clock_gettime(CLOCK_MONOTONIC, &t_end);
// Calculate the time in milliseconds.
double rtt = diff_ms(t_start, t_end);

// Parse IP header to find TTL and ICMP header offset
struct iphdr *ip = (struct iphdr *)recvbuf;
int ip_header_len = ip->ihl * 4;
int ttl = ip->ttl;

// check that the reaceved data is valid
if (n < ip_header_len + (ssize_t)sizeof(struct icmphdr)) {
    fprintf(stderr, "Received packet too short\n");
    close(sock);
    return 1;
}

// Find and create the ICMP package
struct icmphdr *ricmp = (struct icmphdr *)(recvbuf + ip_header_len);

// Validate that this is our Echo Reply
// Echo Reply type = ICMP_ECHOREPLY (0) AND same ID AND same sequence (using ntohs to convert for B-I to L-I)
if (ricmp->type == ICMP_ECHOREPLY &&
    ntohs(ricmp->un.echo.id) == my_id &&
    ntohs(ricmp->un.echo.sequence) == my_seq) {

    // Convert the ip form binary to IP str:
    char from_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &from.sin_addr, from_ip, sizeof(from_ip));

    // Print the <msg>: "64 bytes from X: icmp_seq=1 ttl=117 time=5.98ms"
    printf("%zd bytes from %s: icmp_seq=%u ttl=%d time=%.3fms\n",
           sent, from_ip, my_seq, ttl, rtt);
} else {
    // Could be another ICMP message; for now just show a debug line
    printf("[debug] got ICMP type=%d code=%d (not our echo reply)\n",
           ricmp->type, ricmp->code);
}

close(sock);
return 0;
```

We now modify the code so that more than one request can be sent. Instead of sending a single Echo Request, we add a main loop that sends multiple ICMP requests sequentially. The loop works as follows:

- If the c flag is not provided, the loop runs indefinitely until the user exits with CTRL+C. We add support for graceful termination so that the program does not exit before properly closing the socket; for this we use a signal as described in the assignment appendix.
- If the c flag is provided with a number, the loop runs the number of times specified by the user.

In each iteration of the loop, one request is sent with an increasing sequence number (icmp_seq).

Because we now send multiple ICMP requests rather than a single request, we create a dedicated function for packet construction. This function is called on every loop iteration using the same identifier and an updated sequence number, while the checksum is recalculated accordingly:

```c
// Build packet per-seq (checksum must be recomputed each time)
static void build_icmp_packet(unsigned char packet[ICMP_PACKET_SIZE],
                              unsigned short my_id,
                              unsigned short my_seq)
{
    memset(packet, 0, ICMP_PACKET_SIZE);

    struct icmphdr *icmp = (struct icmphdr *)packet;
    icmp->type = ICMP_ECHO;
    icmp->code = 0;

    icmp->un.echo.id = htons(my_id);
    icmp->un.echo.sequence = htons(my_seq);

    for (int i = 0; i < ICMP_PAYLOAD_SIZE; i++) {
        packet[sizeof(struct icmphdr) + i] = (unsigned char)('A' + (i % 26));
    }

    icmp->checksum = 0;
    icmp->checksum = checksum(packet, ICMP_PACKET_SIZE);
}
```

We also handle the F flag received from the user:

- If the flag is not present, we always pause for one second between requests using sleep(1).

- If the flag is present, packets are sent continuously without any delay.

As noted, we use a signal so that CTRL+C can be handled correctly, including properly closing the socket. We therefore create the following helper function and Boolean flag:

```c
// Stop flag for Ctrl+C (SIGINT)
static volatile sig_atomic_t g_stop = 0;

// Signal handler (do NOT print here)
static void on_sigint(int signo) {
    (void)signo;
    g_stop = 1;
}
```

```c
// Install SIGINT handler (Ctrl+C)
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_handler = on_sigint;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);
```

```c
// Install SIGINT handler (Ctrl+C)
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_handler = on_sigint;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);
```

We define a signal handler for SIGINT. When CTRL+C is pressed, the handler sets a stop flag (g_stop), and the main loop checks this flag and exits the loop and the program in an orderly manner.

We also add the following variables to collect statistics for the ping session (number sent, number received, RTT), as well as the session start time using CLOCK_MONOTONIC so that the total execution time can be calculated:

```c
// Session statistics
long transmitted = 0;
long received = 0;

double rtt_min = 0.0;
double rtt_max = 0.0;
double rtt_sum = 0.0;

// Session timer for total time
struct timespec session_start;
clock_gettime(CLOCK_MONOTONIC, &session_start);

unsigned short my_id = (unsigned short)(getpid() & 0xFFFF);
unsigned short my_seq = 1;

// Reusable buffers
unsigned char packet[ICMP_PACKET_SIZE];
unsigned char recvbuf[RECV_BUF_SIZE];
```

We also create a variable to store the identifier of our ping requests, my_id, and another variable for the request sequence number, my_seq. This lets us associate each received reply with the request that was sent.

For repeated sending and receiving, we need both a send buffer and a receive buffer. We created two fixed reusable buffers for packet construction and reply extraction on every loop iteration, avoiding repeated memory allocation.

We construct a main loop and place inside it the sending and validation logic previously implemented for a single packet, integrating the parameter-related changes described

above:

```
// Main loop - send multiple pings
while (!g_stop && (count == -1 || transmitted < count)) {

    // Build packet for this seq (checksum changes per seq)
    build_icmp_packet(packet, my_id, my_seq);

    // Start counting time for RTT
    struct timespec t_start, t_end;
    clock_gettime(CLOCK_MONOTONIC, &t_start);

    // Send the packet to the destination
    ssize_t sent = sendto(sock, packet, sizeof(packet), 0,
                          (struct sockaddr *)&dest, sizeof(dest));
    if (sent < 0) {
        perror("sendto");
        break;
    }

    transmitted++;

    // Wait up to 10 seconds for a reply using poll()
    struct pollfd pfd;
    memset(&pfd, 0, sizeof(pfd));
    pfd.fd = sock;
    pfd.events = POLLIN;

    int pr = poll(&pfd, 1, TIMEOUT_MS);
    if (pr == 0) {
        printf("Request timeout for icmp_seq %u\n", my_seq);
        break;   // NOT continue
    }
    if (pr < 0) {
        if (errno == EINTR) {
            // Interrupted by signal, loop will stop via g_stop
            continue;
        }
        // inernal error
        perror("poll");
        break;
    }

    // Receive the raw IP packet (it contains IP header + ICMP message)
    struct sockaddr_in from;
    socklen_t fromlen = sizeof(from);
    ssize_t n = recvfrom(sock, recvbuf, sizeof(recvbuf), 0,
                         (struct sockaddr *)&from, &fromlen);
    if (n < 0) {
        if (errno == EINTR) {
            continue;
        }
        perror("recvfrom");
        break;
    }

    // Close the timmer
    clock_gettime(CLOCK_MONOTONIC, &t_end);
    // Calculate the RTT in milliseconds.
    double rtt = diff_ms(t_start, t_end);

    // Parse IP header to find TTL and ICMP header offset
    struct iphdr *ip = (struct iphdr *)recvbuf;
    int ip_header_len = ip->ihl * 4;
    int ttl = ip->ttl;
```

```
    // check that the reaceved data is valid
    if (n < ip_header_len + (ssize_t)sizeof(struct icmphdr)) {
        fprintf(stderr, "Received packet too short\n");
        // Treat as "not received", continue
        my_seq++;
        if (!flood && !g_stop) sleep(1);
        continue;
    }

    // Find and create the ICMP package
    struct icmphdr *ricmp = (struct icmphdr *)(recvbuf + ip_header_len);

    // Validate that this is our Echo Reply
    // Echo Reply type = ICMP_ECHOREPLY (0) AND same ID AND same sequence (using ntohs to convert for B-E to L-E)
    if (ricmp->type == ICMP_ECHOREPLY &&
        ntohs(ricmp->un.echo.id) == my_id &&
        ntohs(ricmp->un.echo.sequence) == my_seq) {

        received++;

        // Update RTT stats
        if (received == 1) {
            rtt_min = rtt_max = rtt;
        } else {
            if (rtt < rtt_min) rtt_min = rtt;
            if (rtt > rtt_max) rtt_max = rtt;
        }
        rtt_sum += rtt;

        // Convert the ip form binary to IP str:
        char from_ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &from.sin_addr, from_ip, sizeof(from_ip));

        // Print the <msg>: "64 bytes from X: icmp_seq=1 ttl=117 time=5.98ms"
        ssize_t icmp_bytes = n - ip_header_len;

        printf("%zd bytes from %s: icmp_seq=%u ttl=%d time=%.3fms\n", icmp_bytes, from_ip, my_seq, ttl, rtt);
    } else {
    // Could be another ICMP message; for now just show a debug line
    printf("[debug] got ICMP type=%d code=%d (not our echo reply)\n",
           ricmp->type, ricmp->code);
    }

    my_seq++;

    // Flood mode means no sleep
    if (!flood && !g_stop) {
        sleep(1);
    }
}
```

The loop now sends multiple ICMP requests according to the C and F flags. In every iteration, a new Echo Request packet is constructed with a new sequence number and an updated checksum. The start time for RTT measurement is then recorded, and poll is used while waiting for a reply.

Instead of letting the operating system create the IP header automatically, we construct it ourselves so that we can set our own TTL value (64) rather than using the default value of 255, as discussed in the lecture.

In the event of a timeout, the program prints a message, terminates the packet-sending loop, and finally prints the statistics.

The received packet is validated by extracting the TTL from the IP header, locating the ICMP header, filtering replies according to type, id, and sequence, and updating RTT

and packet-loss statistics throughout the run. As before, packets that do not match our request are ignored.

The code now also handles program termination using CTRL+C correctly, so execution stops in a controlled manner.

During execution, the following data is collected:

- Number of packets transmitted.
- Number of packets received.
- RTT values for each Echo Reply.
- Total session execution time.

At the end of the run (or after CTRL+C), the following statistics are printed:

- Number of packets transmitted and received.
- Packet loss percentage (a simple calculation that is convenient to inspect at the end of transmission).
- Total time.
- Minimum, average, and maximum RTT.

Finally, we print the data in a simple format:

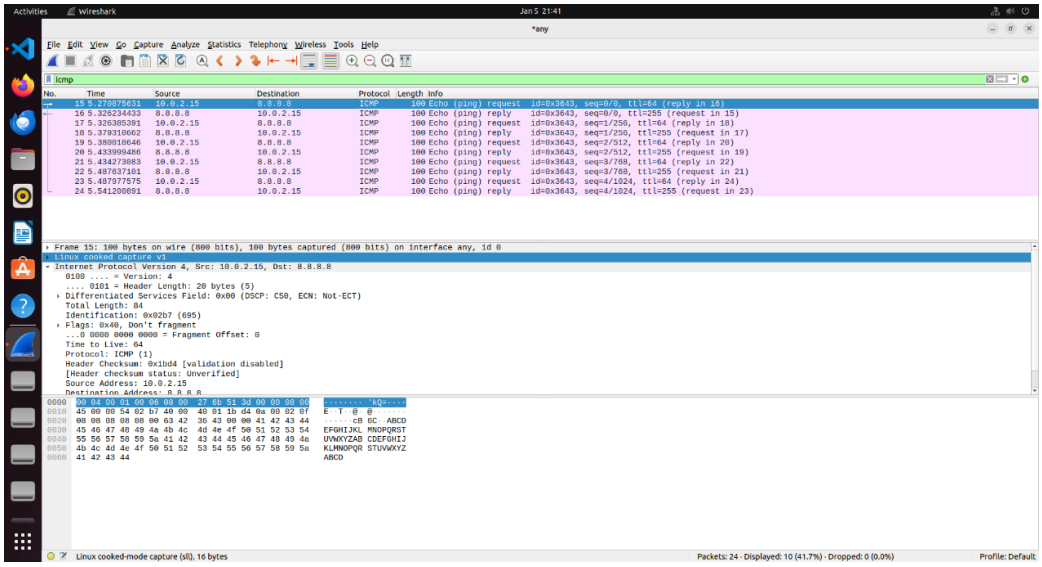
```
// Print final statistics
struct timespec session_end;
clock_gettime(CLOCK_MONOTONIC, &session_end);
double total_ms = diff_ms(session_start, session_end);

printf("\n--- %s ping statistics ---\n", address);
printf("%ld packets transmitted, %ld received, ", transmitted, received);

long lost = transmitted - received;
double loss_pct = 0.0;
if (transmitted > 0) {
    loss_pct = (double)lost * 100.0 / (double)transmitted;
}
printf("%.1f%% packet loss, time %.0fms\n", loss_pct, total_ms);

if (received > 0) {
    double rtt_avg = rtt_sum / (double)received;
    printf("rtt min/avg/max = %.3f/%.3f/%.3f ms\n", rtt_min, rtt_avg, rtt_max);
}

close(sock);
return 0;
```

We now monitor the network traffic and run the following cases according to the assignment requirements:

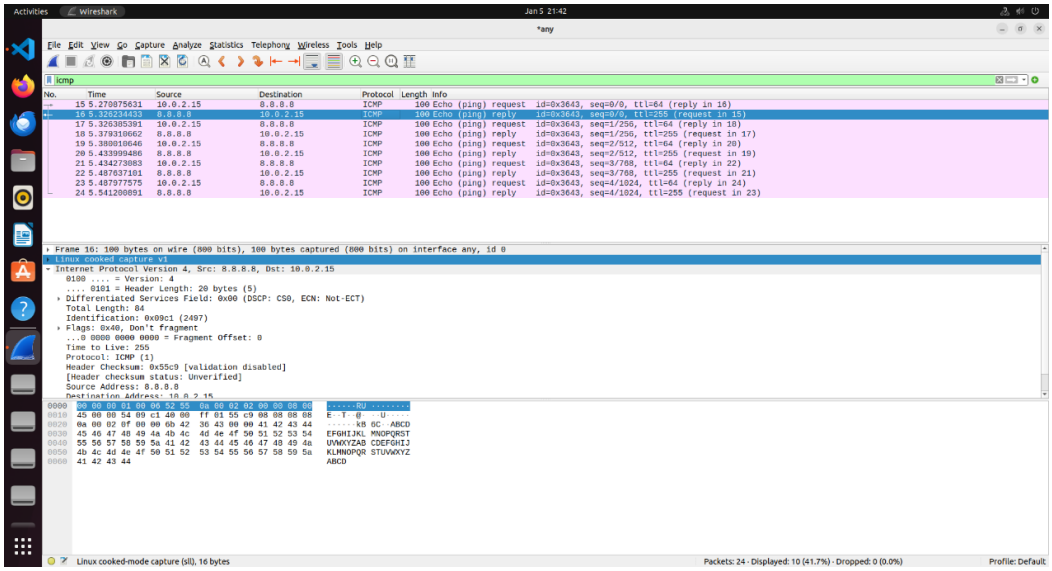
```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.8 -c 5 -f
[sudo] password for vboxuser:
Pinging 8.8.8.8 with 64 bytes of data:
64 bytes from 8.8.8.8: icmp_seq=0 ttl=255 time=55.502ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=53.234ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=54.501ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=53.491ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=53.362ms

--- 8.8.8.8 ping statistics ---
5 packets transmitted, 5 received, 0.0% packet loss, time 271ms
rtt min/avg/max = 53.234/54.018/55.502 ms
```

We can see that the output is correct and the messages were sent as required. We now inspect the network traffic using Wireshark:

The message we send:



The reply we receive:

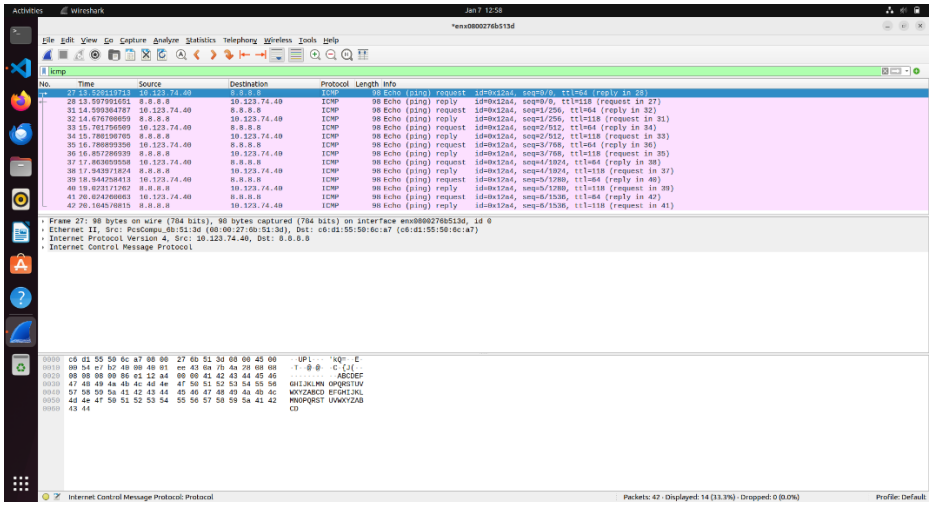


We also verify the remaining cases:

- With no additional flags, only an IP address; we stopped the program using CTRL+C:

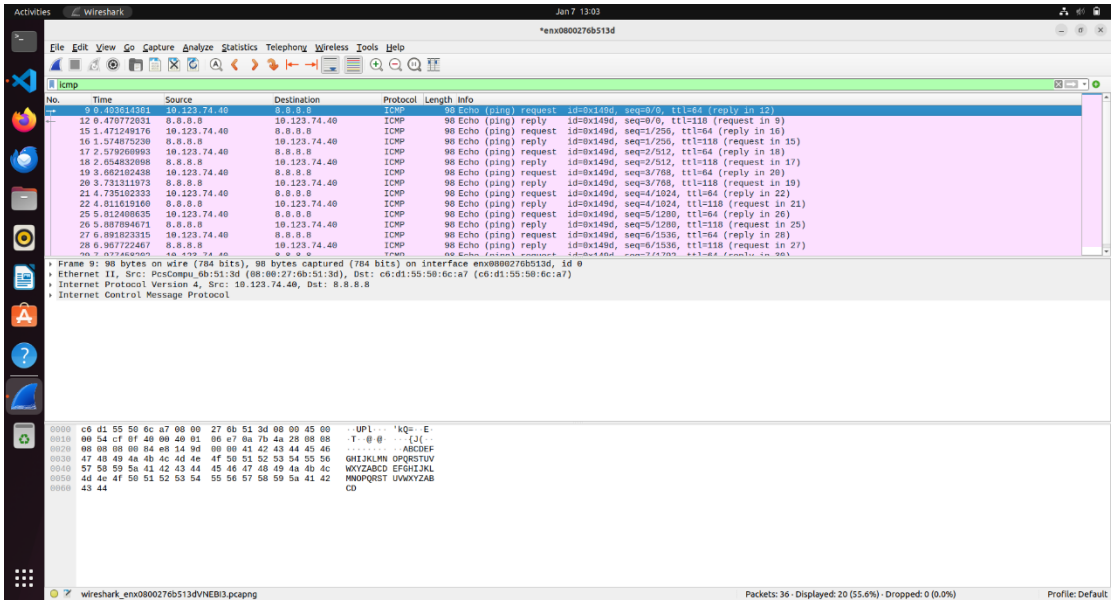
```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.8
[sudo] password for vboxuser:
Pinging 8.8.8.8 with 64 bytes of data:
64 bytes from 8.8.8.8: icmp_seq=0 ttl=118 time=78.095ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=77.551ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=78.620ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=76.652ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=81.105ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=118 time=79.321ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=118 time=80.560ms
^C
--- 8.8.8.8 ping statistics ---
7 packets transmitted, 7 received, 0.0% packet loss, time 7444ms
rtt min/avg/max = 76.652/78.843/81.105 ms
```

We can see all packets being transmitted and received as expected:



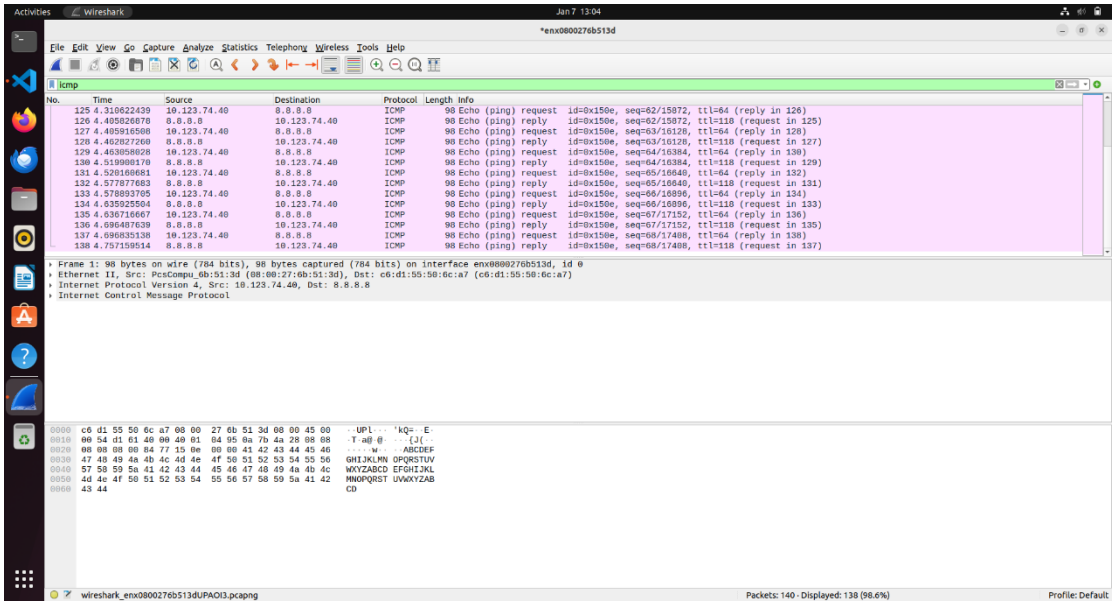
- We now add the C parameter with a value of 10, meaning only 10 messages are sent:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.8 -c 10
Pinging 8.8.8.8 with 64 bytes of data:
64 bytes from 8.8.8.8: icmp_seq=0 ttl=118 time=67.265ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=103.751ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=75.832ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=69.591ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=76.831ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=118 time=75.646ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=118 time=75.985ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=118 time=86.660ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=118 time=73.840ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=118 time=77.713ms

--- 8.8.8.8 ping statistics ---
10 packets transmitted, 10 received, 0.0% packet loss, time 10837ms
rtt min/avg/max = 67.265/78.311/103.751 ms
```



- Next, the F flag sends the messages continuously without delay:

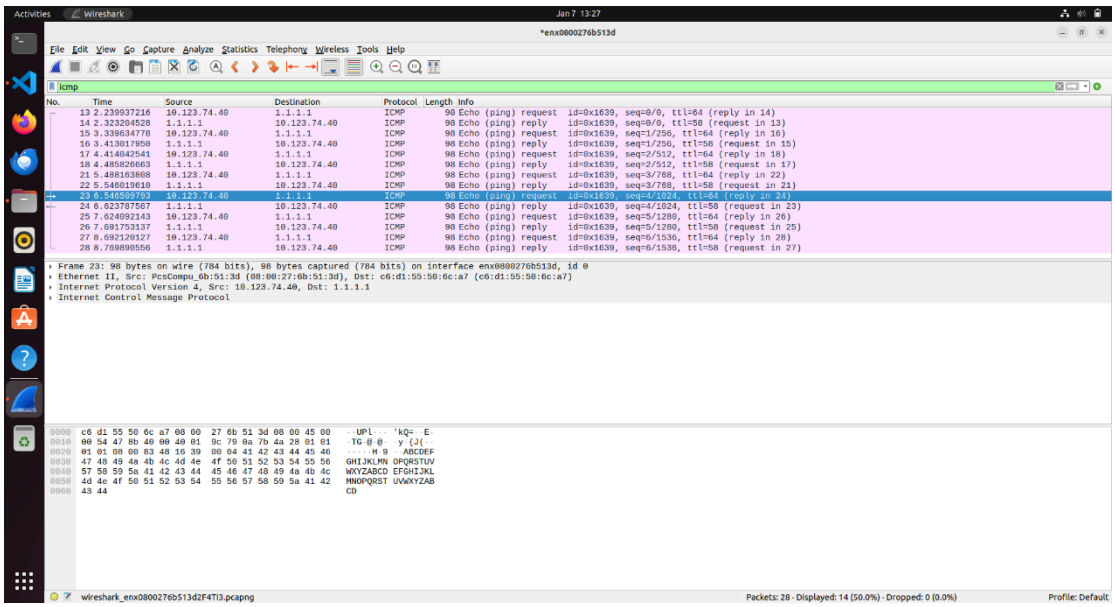
```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.8 -f
64 bytes from 8.8.8.8: icmp_seq=52 ttl=118 time=61.323ms
64 bytes from 8.8.8.8: icmp_seq=53 ttl=118 time=58.028ms
64 bytes from 8.8.8.8: icmp_seq=54 ttl=118 time=98.160ms
64 bytes from 8.8.8.8: icmp_seq=55 ttl=118 time=57.874ms
64 bytes from 8.8.8.8: icmp_seq=56 ttl=118 time=59.872ms
64 bytes from 8.8.8.8: icmp_seq=57 ttl=118 time=57.457ms
64 bytes from 8.8.8.8: icmp_seq=58 ttl=118 time=88.243ms
64 bytes from 8.8.8.8: icmp_seq=59 ttl=118 time=57.312ms
64 bytes from 8.8.8.8: icmp_seq=60 ttl=118 time=61.368ms
64 bytes from 8.8.8.8: icmp_seq=61 ttl=118 time=58.489ms
64 bytes from 8.8.8.8: icmp_seq=62 ttl=118 time=95.270ms
64 bytes from 8.8.8.8: icmp_seq=63 ttl=118 time=57.117ms
64 bytes from 8.8.8.8: icmp_seq=64 ttl=118 time=57.079ms
64 bytes from 8.8.8.8: icmp_seq=65 ttl=118 time=58.712ms
64 bytes from 8.8.8.8: icmp_seq=66 ttl=118 time=57.367ms
64 bytes from 8.8.8.8: icmp_seq=67 ttl=118 time=60.040ms
^C
--- 8.8.8.8 ping statistics ---
69 packets transmitted, 68 received, 1.4% packet loss, time 4745ms
rtt min/avg/max = 57.079/68.865/112.691 ms
```



- For additional variety, we also test another server, this time using a different Cloudflare DNS address:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 1.1.1.1
[sudo] password for vboxuser:
Pinging 1.1.1.1 with 64 bytes of data:
64 bytes from 1.1.1.1: icmp_seq=0 ttl=58 time=83.542ms
64 bytes from 1.1.1.1: icmp_seq=1 ttl=58 time=73.664ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=58 time=72.062ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=58 time=58.095ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=58 time=77.439ms
64 bytes from 1.1.1.1: icmp_seq=5 ttl=58 time=67.754ms
64 bytes from 1.1.1.1: icmp_seq=6 ttl=58 time=77.854ms
^C
--- 1.1.1.1 ping statistics ---
7 packets transmitted, 7 received, 0.0% packet loss, time 6580ms
rtt min/avg/max = 58.095/72.916/83.542 ms
```

Here too, we can capture the packets and verify that everything is transmitted as expected:



We also verify that our parameter validation works correctly and returns an error. In this case, the IP address is invalid:

And now when the a parameter is missing:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping 8.8.8.888 -c 5 -f
Error: -a <address> is required
```

Part B – Traceroute

We now build a program that prints the route taken by an IP packet from our computer to a destination by using ICMP Echo Request packets with increasing TTL values. Whenever the packet reaches an intermediate hop and its TTL expires, an ICMP response is returned to us.

Here too, we use Oracle VirtualBox with Ubuntu 22.04 and a wired Internet connection (Bridged Adapter rather than NAT). This allows the packets to leave the VM directly onto the network so that the intermediate hops on the routed path are actually observed and the TTL values are meaningful. We again use two helper functions from the previous part: the checksum function from the tutorials and diff_ms for RTT calculation.

We begin by reading the parameters from the user. The program accepts one parameter: the destination address. As in Part A, the -a flag is followed by the destination IP address, as required:

sudo ./traceroute -a <destination_ip>

Here too, SUDO privileges are required in order to use a raw socket. If a valid destination parameter is not provided, the program displays an error and terminates.

```c
int main(int argc, char **argv) {
    // destention IP str:
    const char *dest_ip_str = NULL;

    // Get the parameters via the OPT functions:
    int opt;
    while ((opt = getopt(argc, argv, "a:")) != -1) {
        switch (opt) {
            case 'a':
                dest_ip_str = optarg;
                break;
            default:
                // we need 'a', else is not good:
                fprintf(stderr, "no 'a' parameter\n");
                return 1;
        }
    }

    if (!dest_ip_str) {
        // we need 'a'...
        fprintf(stderr, "no 'a' parameter\n");
        return 1;
    }
```

We convert the destination address from a string into an IPv4 structure as required for transmission: the struct is cleared, the address family is set to IPv4, and the address is converted from text to binary. If the IP address is invalid, execution stops with an error message.

We then create a raw socket. As before, we construct the IP header ourselves rather than letting the operating system do so, because we need control over the TTL. The relevant IP-header fields are prepared next:

```c
// Build destination address (standart IPV4)
struct sockaddr_in dest;
memset(&dest, 0, sizeof(dest));
dest.sin_family = AF_INET;

if (inet_pton(AF_INET, dest_ip_str, &dest.sin_addr) != 1) {
    // convert IP to str failed so IP not valid
    fprintf(stderr, "Invalid IPv4 address: %s\n", dest_ip_str);
    return 1;
}

// Create raw socket for sending custom IP packets
int sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
if (sock < 0) {
    // Socket failed
    perror("socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)");
    return 1;
}

// Tell kernel that we include the IP header ourselves
int on = 1;
if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on)) < 0) {
    perror("setsockopt(IP_HDRINCL)");
    close(sock);
    return 1;
}
```

We now construct an ICMP Echo Request packet in memory, similarly to Part A:

- Create a struct icmphdr.
- Set type = ICMP_ECHO.
- Set code = 0.
- Use a fixed identifier (my_id) based on getpid.
- Use a sequence number that increases with every transmission.
- Add a fixed payload.
- Calculate the checksum over the ICMP header and payload.

To identify replies that belong to our requests:

- A my_id identifier is created based on getpid() and limited to 16 bits using a mask.
- A seq counter is maintained and incremented for every ICMP packet sent.

The program also prints an opening line similar to traceroute, containing:

- The destination address.
- The maximum number of hops.
- The number of attempts per hop.

```c
// id helps to tie a response to our request (using the "PID with AND 0xFFFF" for ICMP in 16-bit)
unsigned short my_id = (unsigned short)(getpid() & 0xFFFF);
// seq is raised on each probe, to identify each attempt
unsigned short seq = 1;

// Print log to user:
printf("traceroute to %s, %d hops max, %d probes per hop\n",
       dest_ip_str, MAX_HOPS, PROBES_PER_HOP);
```

We now send all requests while tracking the TTL value. A loop runs from TTL 1 up to MAX_HOPS, which is defined as 30. For each TTL value:

- Print the hop number.
- Send three ICMP Echo Request packets, as required by the assignment.
- Maintain a flag indicating whether the final destination has been reached.

As practiced in the tutorial and required by the assignment, we send multiple probes for every hop. This helps identify packet loss and provides a more stable view of the route.

For every probe, a packet is constructed in memory containing:

- An IP header built using struct iphdr:
  - Version: IPv4.
  - Header length.
  - Total packet length (IP + ICMP).
  - A changing identifier.
  - TTL: the current value in the loop.
  - Protocol: ICMP.
  - Destination address.
  - IP-header checksum calculation.
- An ICMP Echo Request message built using struct icmphdr:
  - type = ICMP_ECHO
  - code = 0
  - identifier = my_id
  - sequence = seq (incremented for every transmission)
  - A fixed 56-byte payload.
  - Checksum calculation over the ICMP header and payload.

After constructing the headers, a single in-memory buffer is produced in the following format:

[IP header][ICMP header][payload]

```c
// Main loop over TTL/hops:
for (int ttl = 1; ttl <= MAX_HOPS; ttl++) {
    // for each hop print the ***
    printf("%2d  ", ttl);
    fflush(stdout); // flush to refresh the screem each hop

    char hop_ip_first[INET_ADDRSTRLEN] = {0}; // Save the first IP returned with the same TTL, to print it once
    int reached_destination = 0; // Stop flag if we reached the destination

    // Each probe is an independent sending of an ICMP Echo Request packet with the same TTL,
    // Sending three probes improves reliability and allows for packet loss detection.
    for (int probe = 0; probe < PROBES_PER_HOP; probe++) {
        // Set a buffer for the packet we send
        unsigned char packet[FULL_PACKET_SIZE];
        memset(packet, 0, sizeof(packet)); // set all the val in the array to 0

        // Build IP header
        struct iphdr *ip = (struct iphdr *)packet;
        ip->version = 4; // IPv4...
        ip->ihl = (unsigned char)(IP_HEADER_SIZE / 4); // number of 32-bit words
        ip->tos = 0;
        ip->tot_len = htons((unsigned short)FULL_PACKET_SIZE);
        ip->id = htons((unsigned short)(my_id + ttl)); // any changing ID is fine
        ip->frag_off = htons(0);
        ip->ttl = (unsigned char)ttl; /// the TTL we send...
        ip->protocol = IPPROTO_ICMP;
        ip->saddr = 0;
        ip->daddr = dest.sin_addr.s_addr;

        ip->check = 0;
        ip->check = checksum(ip, IP_HEADER_SIZE);

        // Build ICMP Echo Request
        struct icmphdr *icmp = (struct icmphdr *)(packet + IP_HEADER_SIZE);
        icmp->type = ICMP_ECHO; // 8
        icmp->code = 0;
        icmp->un.echo.id = htons(my_id);
        icmp->un.echo.sequence = htons(seq++);

        // Payload: can be anything; we fill with a pattern...
        unsigned char *payload = (unsigned char *)(packet + IP_HEADER_SIZE + sizeof(struct icmphdr));
        for (int i = 0; i < ICMP_PAYLOAD_SIZE; i++) {
            payload[i] = (unsigned char)('A' + (i % 26));
        }

        icmp->checksum = 0;
        icmp->checksum = checksum(icmp, ICMP_PACKET_SIZE);
```

For RTT calculation, the start time is recorded using clock_gettime(CLOCK_MONOTONIC) before the packet is sent.

The packet stored in the buffer is then sent using sendto. Afterwards, the program waits for a reply using poll with a one-second timeout.

If no reply is received within the timeout:

- "*" is printed.
- The program continues to the next probe.

```c
// Send probe and measure time
struct timespec t_start, t_end;
clock_gettime(CLOCK_MONOTONIC, &t_start);

ssize_t sent = sendto(sock, packet, FULL_PACKET_SIZE, 0,
                      (struct sockaddr *)&dest, sizeof(dest));
if (sent < 0) {
    // if the send failed we print '*' and try again
    perror("sendto");
    printf("* ");
    continue;
}

// Wait for a reply using poll() with a timeout (non-blocking wait)
struct pollfd pfd;
pfd.fd = sock; // The raw ICMP socket we are listening on
pfd.events = POLLIN; // We are interested only in incoming data

int pr = poll(&pfd, 1, TIMEOUT_MS);
if (pr <= 0) {
    // timeout or error: print '*' and try again
    printf("* ");
    fflush(stdout);
    continue;
}
```

If a packet is received within the allowed time, it is stored, its data is extracted, and its validity is checked:

- First, the RTT is measured.
- The IP header is extracted.
- The ICMP header is extracted.

If an ICMP Echo Reply is received, the final destination has been reached. The identifier is checked against my_id, and if it matches, the destination is marked as reached and the route is complete:

```
// stop the timer and calculator the RTT:
clock_gettime(CLOCK_MONOTONIC, &t_end);
double rtt = diff_ms(&t_start, &t_end);

// Parse outer IP header from received packet:
// Make sure we received at least a full IP header
if (n < (ssize_t)sizeof(struct iphdr)) {
    // not valid IP header so let's try again
    printf("* ");
    fflush(stdout);
    continue;
}

// Parse the outer IP header from the received packet
struct iphdr *rip = (struct iphdr *)recvbuf;
// IP header length is stored in 32-bit words, so multiply by 4
int rip_hlen = rip->ihl * 4;

// Verify that the packet also contains a full ICMP header
if (n < rip_hlen + (ssize_t)sizeof(struct icmphdr)) {
    // to short ICMP header
    printf("* ");
    fflush(stdout);
    continue;
}

// pointer to the ICMP header (right after the IP header)
struct icmphdr *ricmp = (struct icmphdr *)(recvbuf + rip_hlen);

// Convert source IP to string
char hop_ip[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &from.sin_addr, hop_ip, sizeof(hop_ip));

// Determine if this ICMP message corresponds to our probe.
int is_ours = 0;

// Check the ICMP message type we received
if (ricmp->type == ICMP_ECHOREPLY) {
    // Verify that this reply matches our process ID
    if (ntohs(ricmp->un.echo.id) == my_id) {
        is_ours = 1; // This reply belongs to our probe
        reached_destination = 1; // We reached the final destination
    }
```

If an ICMP Time Exceeded message is received, the packet TTL expired at an intermediate router, meaning this router represents a hop along the route.

In all cases, we verify that the reply belongs to the request we sent by checking the ICMP identifier (id). For ICMP error messages, validation is performed against the original embedded ICMP packet. Replies that do not correspond to our requests are discarded.

For valid replies, the RTT is calculated and the hop IP address and delay are printed. If no reply is received for a particular probe, "*" is printed, similar to standard traceroute output.

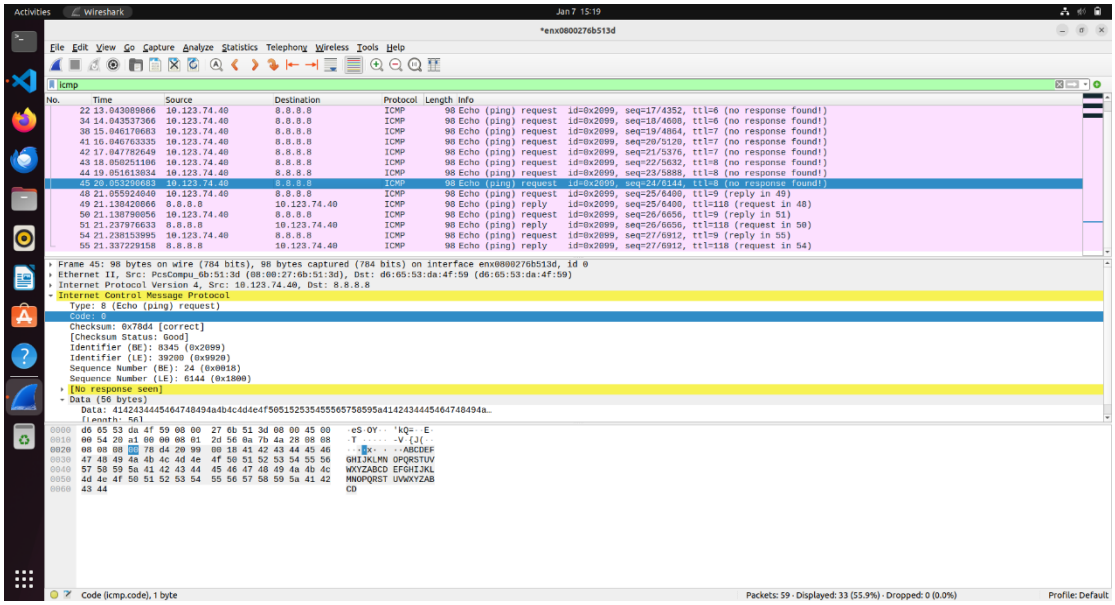
The output for each hop contains the hop number, the router IP address when available, and three RTT values or "*" symbols depending on the replies.

Execution stops when an Echo Reply is received from the destination or when the maximum number of hops (30) is reached. The program then terminates cleanly and closes the socket.

```
        } else if (ricmp->type == ICMP_TIME_EXCEEDED || ricmp->type == ICMP_DEST_UNREACH) {
            // Make sure we have embedded original IP header:
            // Routers send TIME_EXCEEDED (TTL expired) or DEST_UNREACH
            // These ICMP messages contain the *original packet* inside them

            // Point to the embedded (inner) IP header inside the ICMP payload
            unsigned char *inner = (unsigned char *)(recvbuf + rip_hlen + sizeof(struct icmphdr));

            // Length of the embedded data
            ssize_t inner_len = n - (rip_hlen + (ssize_t)sizeof(struct icmphdr));

            // Make sure we have at least an IP header inside
            if (inner_len >= (ssize_t)sizeof(struct iphdr)) {
                // Parse the outer IP header from the received packet
                struct iphdr *inner_ip = (struct iphdr *)inner;
                // IP header length is stored in 32-bit words, so multiply by 4
                int inner_hlen = inner_ip->ihl * 4;

                // Make sure the embedded packet also contains an ICMP header
                if (inner_len >= inner_hlen + (ssize_t)sizeof(struct icmphdr)) {
                    // pointer to the ICMP header (right after the IP header)
                    struct icmphdr *inner_icmp = (struct icmphdr *)(inner + inner_hlen);

                    // Verify that the embedded packet is our ICMP Echo Request
                    if (inner_ip->protocol == IPPROTO_ICMP &&
                        ntohs(inner_icmp->un.echo.id) == my_id) {
                            // This ICMP error refers to our probe
                            is_ours = 1;
                    }
                }
            }
        }

        if (!is_ours) {
            // If the ICMP message is not related to our probe, treat as timeout for this probe.
            printf("* ");
            fflush(stdout);
            continue;
        }

        // Print hop IP once (typical traceroute style), unless it changes
        if (hop_ip_first[0] == '\0') {
            // First response for this hop
            strncpy(hop_ip_first, hop_ip, sizeof(hop_ip_first) - 1);
            hop_ip_first[sizeof(hop_ip_first) - 1] = '\0';
            printf("%s  ", hop_ip_first);
        } else if (strcmp(hop_ip_first, hop_ip) != 0) {
            // Rare, but possible: different routers responding for different probes
            printf("%s  ", hop_ip);
        }

        // Print the round-trip time for this probe
        printf("%.2f ms ", rtt);
        fflush(stdout);
    }


    // End of probes for this TTL
    printf("\n");

    // Stop traceroute once the destination replied
    if (reached_destination) {
        break;
    }
}


// Close the raw socket before exiting
close(sock);
```

We run the program and obtain the expected result:

After the first hop, the program prints several "*" symbols, indicating intermediate routers that do not return ICMP Time Exceeded messages as we might expect, even though packets continue to traverse them. When the TTL reaches 9, an ICMP Echo Reply is received from the destination 8.8.8.8, confirming that the packet reached its destination and that execution can stop. We then analyze the network traffic:



In Wireshark, we can see that valid ICMP Echo Request packets are sent for some TTL values, but no corresponding ICMP response is received. This occurs because some intermediate routers do not send Time Exceeded messages, even when the packet is dropped or otherwise processed. Accordingly, the program prints "*" as explained above.

Part C – Port Scanner

We continue working in the same environment used for the previous two parts. As before, we use code from the tutorials for checksum calculation and for working with IP and TCP header structures.

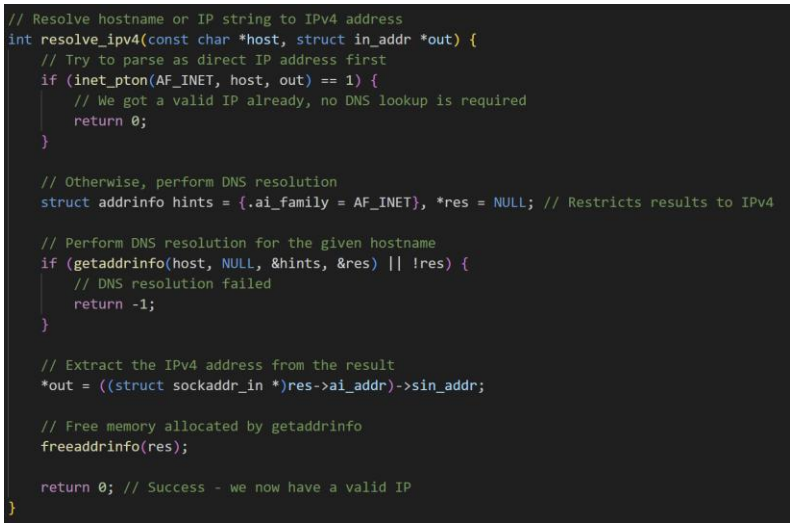
We build a program that scans TCP ports in the range 1 through 65535 and prints which ports are open on a given target. The scan is performed by sending a TCP packet with the SYN flag set and examining the type of response received from the target.

Due to time constraints, this submission supports TCP only.

The program accepts one user parameter: the -a flag followed by a destination address, which may be either an IP address or a hostname. First, the arguments are validated; if a required argument is missing or invalid, the program prints an appropriate error message and terminates. Since only TCP is supported, there is no need for an additional -t flag specifying TCP or UDP.

```c
int main(int argc, char **argv) {
    const char *host = NULL;
    int opt;

    // Parse command line arguments
    while ((opt = getopt(argc, argv, "a:")) != -1) {
        if (opt == 'a') host = optarg;        // Target host
        else {
            fprintf(stderr, "Usage: sudo %s -a <host>\n", argv[0]);
            return 1;
        }
    }

    // Validate required arguments
    if (!host) {
        fprintf(stderr, "Usage: sudo %s -a <host>\n", argv[0]);
        return 1;
    }
```

After receiving the parameters, the destination is converted into a binary IPv4 address. The program first attempts to interpret the string directly as an IP address using inet_pton. If that fails, a DNS lookup is performed using getaddrinfo, restricted to IPv4 results. At the end of this stage, the destination address is available as a struct in_addr. If no valid IP address can be obtained, the program terminates.



```c
// Resolve hostname or IP string to IPv4 address
int resolve_ipv4(const char *host, struct in_addr *out) {
    // Try to parse as direct IP address first
    if (inet_pton(AF_INET, host, out) == 1) {
        // We got a valid IP already, no DNS lookup is required
        return 0;
    }

    // Otherwise, perform DNS resolution
    struct addrinfo hints = {.ai_family = AF_INET}, *res = NULL; // Restricts results to IPv4

    // Perform DNS resolution for the given hostname
    if (getaddrinfo(host, NULL, &hints, &res) || !res) {
        // DNS resolution failed
        return -1;
    }

    // Extract the IPv4 address from the result
    *out = ((struct sockaddr_in *)res->ai_addr)->sin_addr;

    // Free memory allocated by getaddrinfo
    freeaddrinfo(res);

    return 0; // Success - we now have a valid IP
}
```

To construct IP and TCP packets ourselves, we need to know the local IP address from which the computer will send packets to the target. For this purpose, we use a technique in which a UDP socket is opened and connect is called for the destination using a dummy port. Then getsockname is used to retrieve the local IP address selected by the operating system for that route. This operation does not transmit application data but allows us to determine the correct source address.

```c
// Determines the local IPv4 address that would be used to send packets to a given destination
// Uses the "idle UDP socket trick" - no actual packets are sent
int get_local_ip(struct in_addr dst, struct in_addr *src) {
    // Create an IPv4 UDP socket (no packets are actually sent)
    int s = socket(AF_INET, SOCK_DGRAM, 0);
    if (s < 0) return -1;

    // Prepare destination address (IPv4, port 53 DNS as a dummy port)
    struct sockaddr_in d = {
        .sin_family = AF_INET,
        .sin_addr   = dst,
        .sin_port   = htons(53)  // DNS port (arbitrary choice)
    };

    // "Connect" the UDP socket to force route selection
    // This doesn't send any data, just sets up routing
    if (connect(s, (struct sockaddr *)&d, sizeof(d)) < 0) {
        close(s);
        return -1;
    }

    // Length parameter required by getsockname
    socklen_t len = sizeof(d);

    // Query the socket for its locally assigned address
    if (getsockname(s, (struct sockaddr *)&d, &len) < 0) {
        close(s);
        return -1;
    }

    // Extract the local IPv4 address
    *src = d.sin_addr;

    // Close the socket
    close(s);

    return 0; // Success
}
```

Next, two raw sockets are opened: one for sending and one for receiving. For the sending socket, as in the previous sections, the IP_HDRINCL option is enabled so that the operating system does not generate the IP header; instead, we construct it ourselves.

The second socket is used to receive TCP responses, allowing the program to capture SYN+ACK or RST replies from the destination.

SUDO privileges are also required here in order to use both raw sockets.

For checksum calculation of the various headers, we again use the checksum function from the tutorials:

- For IPv4, the checksum is calculated only over the IP header because routers modify IP fields such as TTL along the route.
- For TCP, the checksum is calculated over a pseudo header (source and destination addresses, protocol, and length) together with the TCP header and payload, if present.

```c
// Compute IP header checksum (only header, no payload)
uint16_t ip_checksum(struct iphdr *ip) {
    // Clear the checksum field before calculation
    ip->check = 0;

    // Compute checksum over the IPv4 header (ihl is in 32-bit words)
    return checksum(ip, ip->ihl * 4);
}

// Compute TCP checksum using pseudo header
uint16_t tcp_checksum(const struct iphdr *ip,
                      const struct tcphdr *tcp,
                      size_t len) {

    // Build the TCP pseudo-header required for checksum calculation
    struct pseudo_tcp ph = {
        ip->saddr,       // Source IPv4 address
        ip->daddr,       // Destination IPv4 address
        0,               // Reserved field (must be zero)
        IPPROTO_TCP,     // Protocol identifier for TCP
        htons(len)       // TCP segment length (header + payload)
    };

    // Temporary buffer for pseudo-header + TCP segment
    unsigned char buf[sizeof(ph) + sizeof(struct tcphdr)];

    // Copy pseudo-header into the buffer
    memcpy(buf, &ph, sizeof(ph));

    // Copy TCP header and payload after the pseudo-header
    memcpy(buf + sizeof(ph), tcp, len);

    // Compute and return the checksum over the combined buffer
    return checksum(buf, sizeof(ph) + len);
}
```

Scanning a single port is performed by a dedicated function that constructs a TCP SYN packet in memory. The packet contains an IP header with fixed values such as IPv4, TTL, and source/destination addresses, and a TCP header with the SYN flag set, a random sequence number, and the destination port corresponding to the port being scanned. After the checksums are calculated, the packet is sent to the destination using sendto.

```c
// Create raw socket for sending (with IP_HDRINCL to build our own headers)
int ss = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
if (ss < 0) {
    perror("socket");
    return 1;
}

// Tell kernel we're providing our own IP headers
int one = 1;
if (setsockopt(ss, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one)) < 0) {
    perror("setsockopt");
    close(ss);
    return 1;
}

// Create raw socket for receiving TCP responses
int rs = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
if (rs < 0) {
    perror("socket");
    close(ss);
    return 1;
}

// Use one source port for the whole scan (simplifies matching)
uint16_t sp = 40000 + (getpid() % 20000);

// Scan all ports from 1 to 65535
for (int p = PORT_MIN; p <= PORT_MAX; p++) {
    // Scan the port
    if (scan_tcp(ss, rs, src, dst, sp, p)) {
        // Print if port is open
        printf("OPEN TCP %d\n", p);
        fflush(stdout);  // Immediate output
    }

    // Progress indicator every 2000 ports
    if (p % 2000 == 0)
        fprintf(stderr, "Scanned %d/65535...\n", p);
}

// Clean up sockets
close(rs);
close(ss);

printf("Scan complete.\n");
return 0;
```

After sending the SYN packet, the program waits for a response using poll with a relatively short timeout of 30 milliseconds. If a packet is received, the program verifies that it is indeed a response to the port and packet that were sent. The source address,

destination address, source port, and destination port are checked for a complete match. Only packets satisfying these conditions are processed.

If a TCP response with SYN and ACK flags is received, the port is considered open. The program marks the port as open and prints it. If an RST response is received, the port is considered closed. If no response is received within the timeout, the port is treated as closed for the purposes of this implementation.

A minimal TCP packet with the RST flag is then sent in all cases, whether the port is open, closed, or no response was received.

For an open port that responded with SYN+ACK, an RST packet with the appropriate ACK value is sent to terminate the half-open connection cleanly and avoid leaving the connection pending on the destination side.

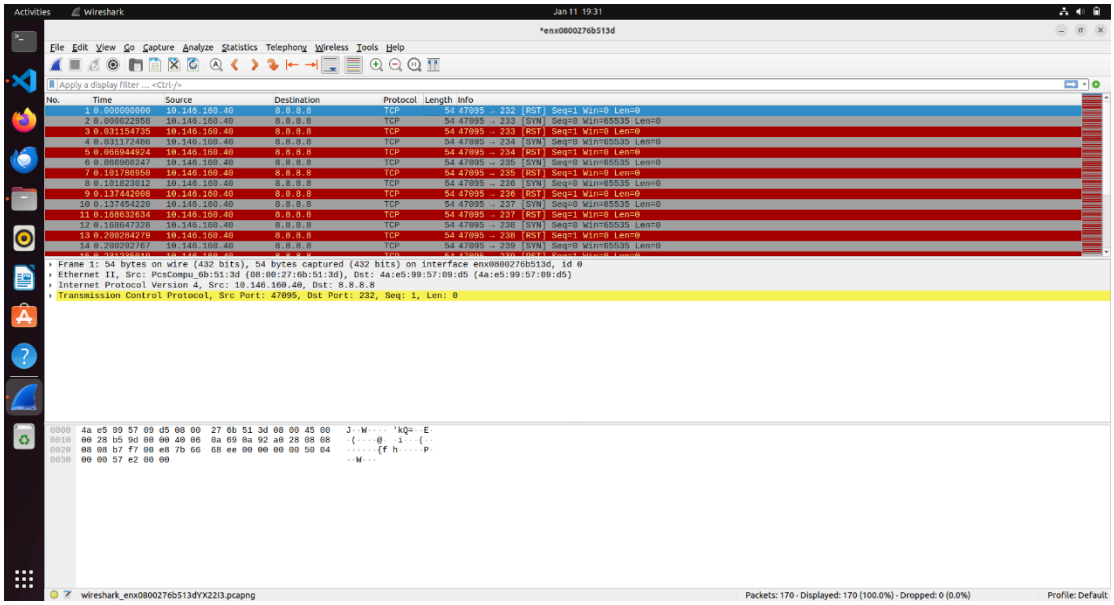
```c
int scan_tcp(int ss, int rs, struct in_addr src, struct in_addr dst, uint16_t sp, uint16_t p) {
    // Allocate packet buffer: IP header + TCP header (no payload)
    unsigned char pkt[sizeof(struct iphdr) + sizeof(struct tcphdr)] = {0};
    struct iphdr *ip = (struct iphdr *)pkt;
    struct tcphdr *tcp = (struct tcphdr *)(pkt + sizeof(struct iphdr));
    uint32_t seq = rand();  // Random initial sequence number

    // Build IP header
    ip->ihl = 5;                        // Header length (5 * 4 = 20 bytes)
    ip->version = 4;                    // IPv4
    ip->ttl = 64;                       // Time to live
    ip->protocol = IPPROTO_TCP;         // TCP protocol
    ip->tot_len = htons(sizeof(pkt));   // Total packet length
    ip->id = htons(rand() & 0xFFFF);    // Random IP ID
    ip->saddr = src.s_addr;             // Source IP (our IP)
    ip->daddr = dst.s_addr;             // Destination IP (target)
    ip->check = ip_checksum(ip);        // Compute IP checksum

    // Build TCP SYN packet
    tcp->source = htons(sp);            // Our source port
    tcp->dest = htons(p);               // Target port
    tcp->seq = htonl(seq);              // Our sequence number
    tcp->doff = 5;                      // Data offset (20 bytes)
    tcp->syn = 1;                       // SYN flag (connection request)
    tcp->window = htons(65535);         // Maximum window size
    tcp->check = tcp_checksum(ip, tcp, sizeof(struct tcphdr));

    // Send the SYN packet
    struct sockaddr_in d = {.sin_family = AF_INET, .sin_addr = dst};
    if (sendto(ss, pkt, sizeof(pkt), 0, (struct sockaddr *)&d, sizeof(d)) < 0) return 0;

    // Wait for response using poll (non-blocking with timeout)
    struct pollfd pfd = {.fd = rs, .events = POLLIN};
    unsigned char buf[2048];
    int open = 0;

    // Poll for TCP_TIMEOUT_MS milliseconds
    if (poll(&pfd, 1, TCP_TIMEOUT_MS) > 0 && (pfd.revents & POLLIN)) {
        // Data is available to read
        ssize_t n = recvfrom(rs, buf, sizeof(buf), 0, NULL, NULL);

        if (n >= (ssize_t)sizeof(struct iphdr)) {
            struct iphdr *rip = (struct iphdr *)buf;
            int ihl = rip->ihl * 4;  // IP header length in bytes

            // Make sure we have enough data for TCP header
            if (n >= ihl + (ssize_t)sizeof(struct tcphdr) && rip->protocol == IPPROTO_TCP) {
                struct tcphdr *rtcp = (struct tcphdr *)(buf + ihl);

                // Verify this response is for our probe
                if (rip->saddr == dst.s_addr &&           // From our target
                    rip->daddr == src.s_addr &&           // To us
                    ntohs(rtcp->source) == p &&           // From the port we scanned
                    ntohs(rtcp->dest) == sp) {            // To our source port

                    uint32_t pseq = ntohl(rtcp->seq);     // Their sequence number
                    uint32_t pack = ntohl(rtcp->ack_seq); // Their ack number

                    // SYN+ACK means port is OPEN
                    if (rtcp->syn && rtcp->ack) {
                        open = 1;
                        // Send RST+ACK to close the connection politely
                        // seq = what they acked, ack = their seq+1
                        send_rst(ss, src, dst, sp, p, pack, pseq + 1, 1);
                    }
                    // RST means port is CLOSED
                    else if (rtcp->rst) {
                        // Still send RST (assignment requirement: send RST either way)
                        send_rst(ss, src, dst, sp, p, pack, pseq, 1);
                    }
                }
            }
        }
    }
```

This mechanism is executed in a loop scanning the entire port range from 1 to 65535. A fixed high-numbered source port is used to simplify matching responses and avoid conflicts with other services and programs running on the computer. Each port is scanned, and open ports are printed. The program also prints a progress message every few thousand ports.

At the end of the scan, the sockets are closed and the program terminates cleanly.

We now inspect the network traffic during execution:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./port_scanning -a 8.8.8.8
Scanning 8.8.8.8 (8.8.8.8) with TCP...
OPEN TCP 53
OPEN TCP 80
```



We can see a sequence of TCP packets leaving our computer toward the destination. Most of the packets are shown in red because they carry the RST flag. This indicates packets used to terminate connections or clean up half-open connections, consistent with the behavior of our SYN scan. A smaller number of packets shown in blue represent regular TCP packets such as SYN packets.